

ELT para Snowflake Schema - empresa_elt

Flujo: EXTRACT -> LOAD -> TRANSFORM -> Snowflake Schema + KPIs

¿Qué es ELT?

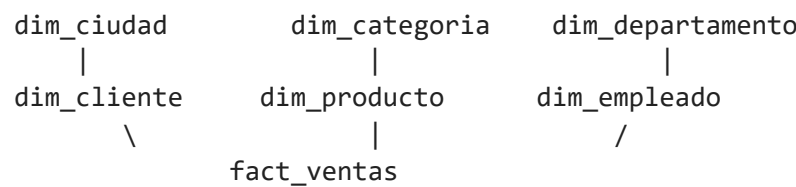
Fase	Descripción
Extract	Carga los datos crudos desde la fuente
Load	Carga los datos sin limpiar directamente a la base de datos destino
Transform	Limpia y normaliza los datos dentro de la base de datos

Diferencia con ETL: En ELT los datos se cargan primero y se transforman después, aprovechando el motor de la base de datos para las transformaciones.

¿Qué es un Snowflake Schema?

Un esquema en copo de nieve es una extensión del Star Schema donde las dimensiones están normalizadas en sub-dimensiones:

- dim_cliente → depende de → dim_ciudad
- dim_producto → depende de → dim_categoria
- dim_empleado → depende de → dim_departamento
- fact_ventas → centro que une todas las dimensiones



Característica	Star Schema	Snowflake Schema
Dimensiones normalizadas	No	Sí
Tablas	Menos	Más
Redundancia	Alta	Baja
Consultas	Más simples	Requieren más JOINS

Conexión a MySQL

```
In [1]: %load_ext sql

usuario      = 'Daniella'
contrasena   = 'daniella123'
host         = '127.0.0.1'
puerto      = '3306'

%sql mysql+pymysql://Daniella:daniella123@127.0.0.1:3306/
```

Crear la base de datos empresa_elt

```
In [2]: %%sql
DROP DATABASE IF EXISTS empresa_elt;

CREATE DATABASE empresa_elt
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
15 rows affected.
1 rows affected.
```

Out[2]: []

```
In [3]: %%sql
USE empresa_elt;
SET SQL_SAFE_UPDATES = 0;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
0 rows affected.
```

Out[3]: []

1. EXTRACT - Tablas Staging

En ELT los datos se **extraen tal como están** desde la fuente hacia tablas staging. No se aplica ninguna transformación aquí — los datos pueden tener NULLs y duplicados.

Problema	Tabla
cliente_id = 4 duplicado	stg_clientes
nombre = NULL	stg_clientes, stg_empleados
producto = NULL	stg_productos

```
In [4]: %%sql
CREATE TABLE stg_clientes (
    cliente_id INT,
    nombre      VARCHAR(100),
    ciudad      VARCHAR(100),
    edad        INT
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[4]: []

```
In [5]: %%sql
INSERT INTO stg_clientes VALUES
(1, 'Carlos', 'San Salvador', 25),
(2, 'Ana',    'Santa Ana',    30),
(3, NULL,    'San Miguel',    22),
(4, 'Pedro', 'La Libertad', 40),
(4, 'Pedro', 'La Libertad', 40);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[5]: []

```
In [6]: %%sql
CREATE TABLE stg_productos (
    producto_id INT,
    producto     VARCHAR(100),
    categoria    VARCHAR(100),
    precio       DECIMAL(10,2)
);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[6]: []

```
In [7]: %%sql
INSERT INTO stg_productos VALUES
(1, 'Laptop', 'Tecnologia', 1200.00),
(2, 'Mouse',  'Tecnologia',  25.00),
(3, 'Silla',  'Muebles',    150.00),
(4, NULL,    'Tecnologia',  75.00);

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[7]: []

```
In [8]: %%sql
CREATE TABLE stg_empleados (
    empleado_id INT,
    nombre       VARCHAR(100),
    departamento VARCHAR(100)
);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[8]: []

```
In [9]: %%sql
INSERT INTO stg_empleados VALUES
(1, 'Maria', 'Ventas'),
(2, 'Jose', 'Ventas'),
(3, NULL, 'Marketing');
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[9]: []

```
In [10]: %%sql
CREATE TABLE stg_ventas (
    venta_id INT,
    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT
);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
```

Out[10]: []

```
In [11]: %%sql
INSERT INTO stg_ventas VALUES
(1, 1, 1, 1, 2),
(2, 2, 2, 1, 5),
(3, 3, 3, 2, 1),
(4, 4, 1, 2, 1),
(5, 1, 2, 1, 3);
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[11]: []

Ver datos crudos en staging

```
In [12]: %%sql
SELECT 'stg_clientes' AS tabla, COUNT(*) AS filas FROM stg_clientes
UNION ALL
SELECT 'stg_productos', COUNT(*) FROM stg_productos
UNION ALL
SELECT 'stg_empleados', COUNT(*) FROM stg_empleados
UNION ALL
SELECT 'stg_ventas', COUNT(*) FROM stg_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[12]:

tabla	filas
stg_clientes	5
stg_productos	4
stg_empleados	3
stg_ventas	5

```
In [13]: %%sql
SELECT * FROM stg_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[13]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

```
In [14]: %%sql
SELECT * FROM stg_productos;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[14]:

producto_id	producto	categoria	precio
1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	None	Tecnologia	75.00

```
In [15]: %%sql
SELECT * FROM stg_empleados;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[15]:

empleado_id	nombre	departamento
1	Maria	Ventas
2	Jose	Ventas
3	None	Marketing

```
In [16]: %%sql
SELECT * FROM stg_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

```
Out[16]: venta_id cliente_id producto_id empleado_id cantidad
```

1	1	1	1	2
2	2	2	1	5
3	3	3	2	1
4	4	1	2	1
5	1	2	1	3

2. LOAD - Cargar datos a tablas ELT

En ELT los datos se cargan sin transformar a las tablas destino. La limpieza vendrá después con UPDATE directamente sobre estas tablas.

Esta es la diferencia con ETL: primero cargamos, luego transformamos dentro de la base de datos.

```
In [17]: %%sql
CREATE TABLE elt_clientes AS
SELECT * FROM stg_clientes;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

```
Out[17]: []
```

```
In [18]: %%sql
CREATE TABLE elt_productos AS
SELECT * FROM stg_productos;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

```
Out[18]: []
```

```
In [19]: %%sql
CREATE TABLE elt_empleados AS
SELECT * FROM stg_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

```
Out[19]: []
```

```
In [20]: %%sql
-- La tabla de ventas se crea con columnas extra para precio y total
-- que se calcularán en la fase de Transform
CREATE TABLE elt_ventas (
    venta_id INT,
    cliente_id INT,
    producto_id INT,
    empleado_id INT,
    cantidad INT,
    precio DECIMAL(10,2),
    total DECIMAL(10,2)
);

INSERT INTO elt_ventas (venta_id, cliente_id, producto_id, empleado_id, cantidad)
SELECT venta_id, cliente_id, producto_id, empleado_id, cantidad
FROM stg_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
5 rows affected.

Out[20]: []

Verificar carga inicial

```
In [21]: %%sql
-- Las tablas ELT tienen los mismos datos crudos (con NULLs y duplicados)
SELECT * FROM elt_clientes;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

Out[21]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	None	San Miguel	22
4	Pedro	La Libertad	40
4	Pedro	La Libertad	40

```
In [22]: %%sql
-- precio y total aún son NULL, se calculan en Transform
SELECT * FROM elt_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.

```
Out[22]:
```

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
1	1	1	1	2	None	None
2	2	2	1	5	None	None
3	3	3	2	1	None	None
4	4	1	2	1	None	None
5	1	2	1	3	None	None

3. TRANSFORM - Limpieza y enriquecimiento dentro de la BD

Las transformaciones se aplican **con UPDATE y recreación de tablas directamente** sobre los datos ya cargados.

Problema	Tabla	Solución SQL
nombres NULL en clientes	elt_clientes	UPDATE ... SET nombre = 'Sin Nombre'
Filas duplicadas en clientes	elt_clientes	DISTINCT + DROP + RENAME
producto NULL	elt_productos	UPDATE ... SET producto = 'Producto Desconocido'
nombre NULL en empleados	elt_empleados	UPDATE ... SET nombre = 'Sin Nombre'
precio y total vacíos	elt_ventas	UPDATE + JOIN con elt_productos

Limpiar nombres NULL en clientes

```
In [23]: %%sql
UPDATE elt_clientes
SET nombre = 'Sin Nombre'
WHERE nombre IS NULL;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

```
Out[23]: []
```

Eliminar duplicados en clientes

```
In [24]: %%sql
-- Crear tabla temporal sin duplicados
CREATE TABLE clientes_temp AS
SELECT DISTINCT *
FROM elt_clientes;
```



```
-- Reemplazar la tabla original
DROP TABLE elt_clientes;
RENAME TABLE clientes_temp TO elt_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
0 rows affected.
0 rows affected.
```

Out[24]: []

```
In [25]: %%sql
-- Antes: 5 filas con duplicado y NULL | Después: 4 filas limpias
SELECT * FROM elt_clientes;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[25]:

cliente_id	nombre	ciudad	edad
1	Carlos	San Salvador	25
2	Ana	Santa Ana	30
3	Sin Nombre	San Miguel	22
4	Pedro	La Libertad	40

Limpiar nombres NULL en productos

```
In [26]: %%sql
UPDATE elt_productos
SET producto = 'Producto Desconocido'
WHERE producto IS NULL;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[26]: []

```
In [27]: %%sql
SELECT * FROM elt_productos;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[27]:

producto_id	producto	categoria	precio
1	Laptop	Tecnologia	1200.00
2	Mouse	Tecnologia	25.00
3	Silla	Muebles	150.00
4	Producto Desconocido	Tecnologia	75.00

Limpiar nombres NULL en empleados

```
In [28]: %%sql
UPDATE elt_empleados
SET nombre = 'Sin Nombre'
WHERE nombre IS NULL;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.
```

Out[28]: []

```
In [29]: %%sql
SELECT * FROM elt_empleados;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[29]: **empleado_id nombre departamento**

1	Maria	Ventas
2	Jose	Ventas
3	Sin Nombre	Marketing

Calcular métricas de ventas (precio y total)

```
In [30]: %%sql
UPDATE elt_ventas v
JOIN elt_productos p
ON v.producto_id = p.producto_id
SET
v.precio = p.precio,
v.total = v.cantidad * p.precio;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[30]: []

```
In [31]: %%sql
-- precio y total ahora están calculados
SELECT * FROM elt_ventas;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
5 rows affected.
```

Out[31]: **venta_id cliente_id producto_id empleado_id cantidad precio total**

1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

4. NORMALIZACIÓN - Crear sub-dimensiones

Esta es la característica clave del Snowflake Schema: las dimensiones se normalizan extrayendo atributos repetidos a tablas separadas.

Sub-dimensión	Extraída de	Propósito
dim_ciudad	elt_clientes.ciudad	Evitar repetir el nombre de ciudad en cada cliente
dim_categoria	elt_productos.categoria	Evitar repetir la categoría en cada producto
dim_departamento	elt_empleados.departamento	Evitar repetir el departamento en cada empleado

dim_ciudad

```
In [32]: %%sql
CREATE TABLE dim_ciudad (
    ciudad_id INT AUTO_INCREMENT PRIMARY KEY,
    ciudad VARCHAR(100) UNIQUE
);

INSERT INTO dim_ciudad (ciudad)
SELECT DISTINCT ciudad
FROM elt_clientes;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.

Out[32]: []

```
In [33]: %%sql
SELECT * FROM dim_ciudad;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

Out[33]:

ciudad_id	ciudad
4	La Libertad
3	San Miguel
1	San Salvador
2	Santa Ana

dim_categoria

```
In [34]: %%sql
CREATE TABLE dim_categoria (
```

```

        categoria_id INT AUTO_INCREMENT PRIMARY KEY,
        categoria    VARCHAR(100) UNIQUE
    );

INSERT INTO dim_categoria (categoria)
SELECT DISTINCT categoria
FROM elt_productos;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
2 rows affected.

```

Out[34]: []

```

In [35]: %%sql
SELECT * FROM dim_categoria;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.

```

Out[35]: **categoria_id** **categoria**

2	Muebles
---	---------

1	Tecnologia
---	------------

dim_departamento

```

In [36]: %%sql
CREATE TABLE dim_departamento (
    departamento_id INT AUTO_INCREMENT PRIMARY KEY,
    departamento    VARCHAR(100) UNIQUE
);

INSERT INTO dim_departamento (departamento)
SELECT DISTINCT departamento
FROM elt_empleados;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
2 rows affected.

```

Out[36]: []

```

In [37]: %%sql
SELECT * FROM dim_departamento;

```

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.

```

Out[37]: **departamento_id** **departamento**

2	Marketing
---	-----------

1	Ventas
---	--------

5. CREACIÓN DE DIMENSIONES - Con FK a sub-dimensiones

Las dimensiones principales ahora referencian a sus sub-dimensiones mediante FOREIGN KEY. Esto elimina la redundancia: en lugar de repetir 'San Salvador' en cada cliente, solo se guarda el `ciudad_id`.

dim_cliente → referencia dim_ciudad

```
In [38]: %%sql
CREATE TABLE dim_cliente (
    cliente_id INT PRIMARY KEY,
    nombre     VARCHAR(100),
    edad       INT,
    ciudad_id  INT,

    FOREIGN KEY (ciudad_id)
        REFERENCES dim_ciudad(ciudad_id)
);

INSERT INTO dim_cliente (cliente_id, nombre, edad, ciudad_id)
SELECT
    c.cliente_id,
    c.nombre,
    c.edad,
    ci.ciudad_id
FROM elt_clientes c
JOIN dim_ciudad ci
    ON c.ciudad = ci.ciudad;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[38]: []

```
In [39]: %%sql
SELECT * FROM dim_cliente;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[39]: **cliente_id nombre edad ciudad_id**

1	Carlos	25	1
2	Ana	30	2
3	Sin Nombre	22	3
4	Pedro	40	4

dim_producto → referencia dim_categoria

```
In [40]: %%sql
CREATE TABLE dim_producto (
    producto_id INT PRIMARY KEY,
    producto VARCHAR(100),
    precio DECIMAL(10,2),
    categoria_id INT,

    FOREIGN KEY (categoria_id)
        REFERENCES dim_categoria(categoria_id)
);

INSERT INTO dim_producto (producto_id, producto, precio, categoria_id)
SELECT
    p.producto_id,
    p.producto,
    p.precio,
    c.categoria_id
FROM elt_productos p
JOIN dim_categoria c
ON p.categoria = c.categoria;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
4 rows affected.
```

Out[40]: []

```
In [41]: %%sql
SELECT * FROM dim_producto;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.
```

Out[41]:

producto_id	producto	precio	categoria_id
1	Laptop	1200.00	1
2	Mouse	25.00	1
3	Silla	150.00	2
4	Producto Desconocido	75.00	1

dim_empleado → referencia dim_departamento

```
In [42]: %%sql
CREATE TABLE dim_empleado (
    empleado_id INT PRIMARY KEY,
    nombre VARCHAR(100),
    departamento_id INT,

    FOREIGN KEY (departamento_id)
        REFERENCES dim_departamento(departamento_id)
);

INSERT INTO dim_empleado (empleado_id, nombre, departamento_id)
```

```
SELECT
    e.Empleado_id,
    e.nombre,
    d.departamento_id
FROM elt_empleados e
JOIN dim_departamento d
    ON e.departamento = d.departamento;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
0 rows affected.
3 rows affected.
```

Out[42]: []

```
In [43]: %%sql
SELECT * FROM dim_Empleado;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/
3 rows affected.
```

Out[43]: **Empleado_id nombre departamento_id**

1	Maria	1
2	Jose	1
3	Sin Nombre	2

6. TABLA DE HECHOS - fact_ventas

La tabla de hechos es el centro del Snowflake Schema. Contiene las métricas medibles y las **Foreign Keys** que apuntan a las dimensiones principales.

Nota: fact_ventas no apunta directamente a dim_ciudad, dim_categoria ni dim_departamento. Para llegar a ellas se llega a través de las dimensiones principales.

```
In [44]: %%sql
CREATE TABLE fact_ventas (
    venta_id    INT PRIMARY KEY,
    cliente_id  INT,
    producto_id INT,
    empleado_id INT,
    cantidad    INT,
    precio      DECIMAL(10,2),
    total       DECIMAL(10,2),

    FOREIGN KEY (cliente_id) REFERENCES dim_cliente(cliente_id),
    FOREIGN KEY (producto_id) REFERENCES dim_producto(producto_id),
    FOREIGN KEY (empleado_id) REFERENCES dim_empleado(empleado_id)
);
```

```
INSERT INTO fact_ventas  
SELECT * FROM elt_ventas;
```

```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
0 rows affected.  
5 rows affected.
```

Out[44]: []

```
In [45]: %%sql  
SELECT * FROM fact_ventas;
```

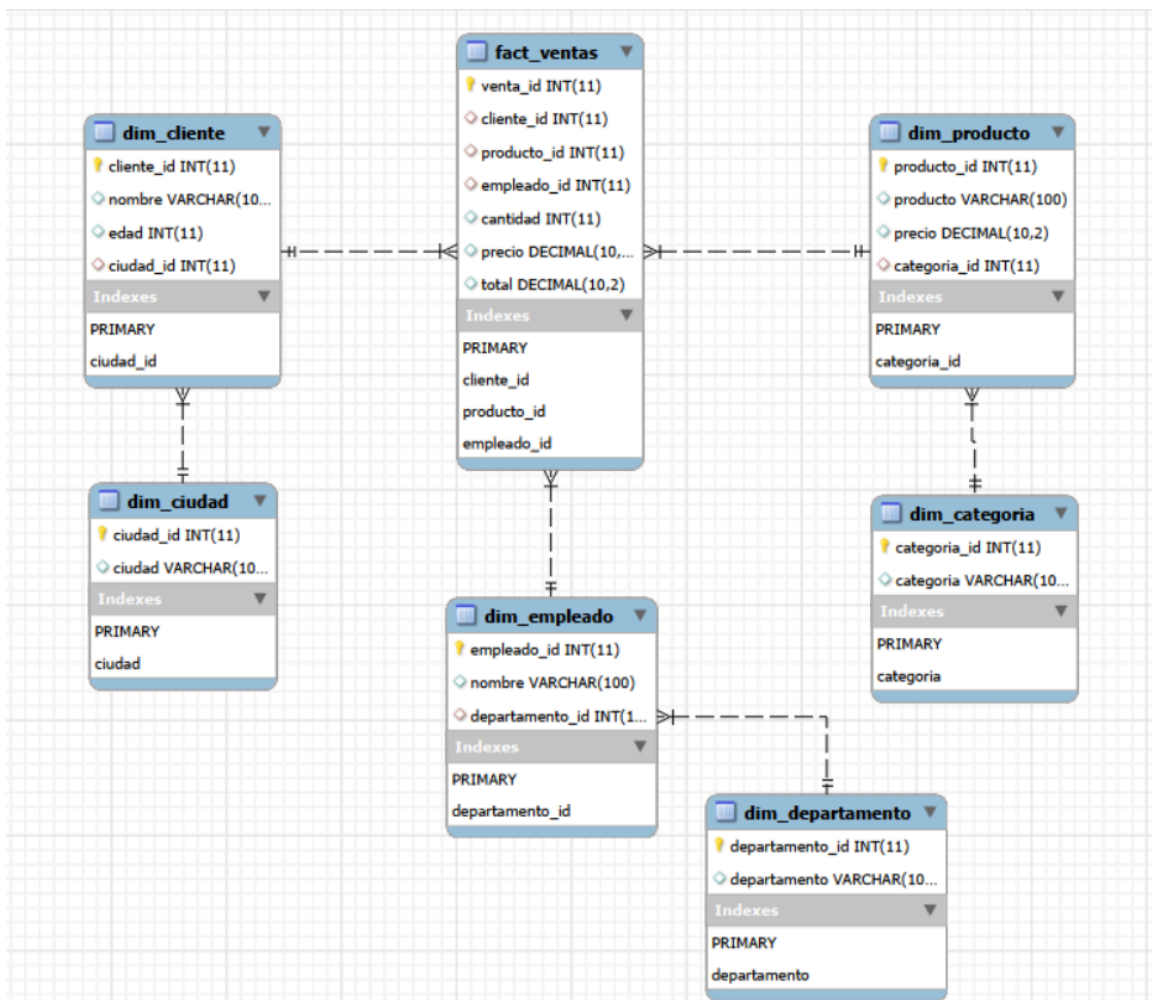
```
* mysql+pymysql://Daniella:***@127.0.0.1:3306/  
5 rows affected.
```

Out[45]:

venta_id	cliente_id	producto_id	empleado_id	cantidad	precio	total
----------	------------	-------------	-------------	----------	--------	-------

1	1	1	1	2	1200.00	2400.00
2	2	2	1	5	25.00	125.00
3	3	3	2	1	150.00	150.00
4	4	1	2	1	1200.00	1200.00
5	1	2	1	3	25.00	75.00

7. Diagrama del Snowflake Schema



Tenemos 3 niveles de tablas:

fact_ventas: es la tabla más importante. Guarda cada venta que ocurrió: cuánto se vendió, a qué precio y el total. No repite información solo guarda los IDs que apuntan a las demás tablas.

las dimensiones principales: son dim_cliente, dim_producto y dim_empleado. Describen quién compró, qué se vendió y quién vendió. Aquí está la diferencia con el Star Schema: estas tablas no guardan directamente la ciudad, la categoría o el departamento, solo guardan un ID que apunta al siguiente nivel.

las sub-dimensiones: son dim_ciudad, dim_categoria y dim_departamento. Aquí es donde vive el dato normalizado. Por ejemplo, "San Salvador" solo se escribe una vez en dim_ciudad, y todos los clientes de esa ciudad simplemente apuntan a ese registro para no repetir datos. Si mañana "San Salvador" cambia de nombre, solo se modifica en un lugar, no en cada fila de clientes. Eso se llama normalización, y es la característica clave del Snowflake Schema.

8. KPIs - Consultas analíticas

En el Snowflake Schema los JOINS son más largos porque hay que navegar por las sub-dimensiones para llegar a los atributos normalizados.

KPI 1 - Ingreso total y promedio por venta

```
In [46]: %%sql
SELECT
    COUNT(*) AS num_ventas,
    SUM(total) AS ingreso_total,
    AVG(total) AS promedio_venta
FROM fact_ventas;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
1 rows affected.

```
Out[46]: num_ventas ingreso_total promedio_venta
```

5	3950.00	790.000000
---	---------	------------

KPI 2 - Ventas por ciudad

En Snowflake Schema hay que hacer JOIN doble: fact_ventas → dim_cliente → dim_ciudad

```
In [47]: %%sql
SELECT
    ci.ciudad,
    SUM(f.total) AS ventas
FROM fact_ventas f
JOIN dim_cliente c
    ON f.cliente_id = c.cliente_id
JOIN dim_ciudad ci
    ON c.ciudad_id = ci.ciudad_id
GROUP BY ci.ciudad
ORDER BY ventas DESC;
```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
4 rows affected.

```
Out[47]: ciudad ventas
```

San Salvador	2475.00
La Libertad	1200.00
San Miguel	150.00
Santa Ana	125.00

KPI 3 - Ingresos por categoría de producto

JOIN doble: fact_ventas → dim_producto → dim_categoria

```
In [48]: %%sql
SELECT
    cat.categoria,
    SUM(f.total) AS ingresos
FROM fact_ventas f
JOIN dim_producto p
    ON f.producto_id = p.producto_id
JOIN dim_categoria cat
    ON p.categoria_id = cat.categoria_id
GROUP BY cat.categoria
ORDER BY ingresos DESC;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.
```

```
Out[48]: categoria  ingresos
Tecnologia      3800.00
Muebles         150.00
```

KPI 4 - Rendimiento de empleados por departamento

JOIN doble: fact_ventas → dim_empleado → dim_departamento

```
In [49]: %%sql
SELECT
    e.nombre      AS empleado,
    d.departamento,
    SUM(f.total)  AS ventas_generadas
FROM fact_ventas f
JOIN dim_empleado e
    ON f.empleado_id = e.empleado_id
JOIN dim_departamento d
    ON e.departamento_id = d.departamento_id
GROUP BY e.nombre, d.departamento
ORDER BY ventas_generadas DESC;

* mysql+pymysql://Daniella:***@127.0.0.1:3306/
2 rows affected.
```

```
Out[49]: empleado  departamento  ventas_generadas
Maria      Ventas      2600.00
Jose       Ventas      1350.00
```

KPI 5 - Top clientes por monto comprado

```
In [50]: %%sql
SELECT
    c.nombre      AS cliente,
    ci.ciudad,
    SUM(f.total)  AS total_comprado
```

```

FROM fact_ventas f
JOIN dim_cliente c
  ON f.cliente_id = c.cliente_id
JOIN dim_ciudad ci
  ON c.ciudad_id = ci.ciudad_id
GROUP BY c.nombre, ci.ciudad
ORDER BY total_comprado DESC;

```

* mysql+pymysql://Daniella:***@127.0.0.1:3306/

4 rows affected.

Out[50]:

cliente	ciudad	total_comprado
Carlos	San Salvador	2475.00
Pedro	La Libertad	1200.00
Sin Nombre	San Miguel	150.00
Ana	Santa Ana	125.00

9. Resumen del proceso ELT + Snowflake Schema

Paso	Fase	Acción
1	Extract	Datos crudos cargados en tablas staging (con NULLs y duplicados)
2	Load	Copia directa de staging a tablas elt_ sin transformar
3	Transform	UPDATE para NULLs, DISTINCT+RENAME para duplicados, JOIN para calcular total
4	Normalización	Sub-dimensiones: dim_ciudad, dim_categoria, dim_departamento
5	Dimensiones	dim_cliente, dim_producto, dim_empleado con FK a sub-dimensiones
6	Fact Table	fact_ventas con FK a dimensiones principales y métricas de negocio
7	KPIs	5 consultas analíticas con gráficos y JOINS por sub-dimensiones

Problemas detectados y resueltos

Problema	Tabla	Solución
Fila duplicada Pedro id=4	stg_clientes	SELECT DISTINCT + DROP + RENAME
nombre = NULL	stg_clientes, stg_empleados	UPDATE ... SET nombre = 'Sin Nombre'
producto = NULL	stg_productos	UPDATE ... SET producto = 'Producto Desconocido'
precio y total vacíos	elt_ventas	UPDATE + JOIN con elt_productos
ciudad repetida en clientes	elt_clientes	Normalizada en dim_ciudad

Problema	Tabla	Solución
categoría repetida en productos	elt_productos	Normalizada en dim_categoria
departamento repetido en empleados	elt_empleados	Normalizado en dim_departamento

ETL vs ELT - ¿Cuál usar?

Criterio	ETL	ELT
¿Dónde se transforma?	Antes de cargar (fuera de BD)	Después de cargar (dentro de BD)
Ideal para	Datos sensibles, recursos limitados	Big Data, bases de datos potentes
Velocidad de carga	Más lenta	Más rápida
Herramientas	Python, Spark	SQL, dbt, Snowflake